COMPLIANCE EVIDENCE INTELLIGENCE

A proposed innovation for SIH Problem Statement 26262

Core idea

Move beyond storing uploaded files. Connect each regulatory requirement to the evidence intended to satisfy it, evaluate what can be checked automatically, and explain what remains missing, expired, inconsistent, or awaiting authorised verification.

At a glance	What the module provides
Purpose	Evidence-based compliance status for application and post-approval obligations.
Primary users	Institutions, reviewing officers, compliance officers, supervisors and auditors—subject to role permissions.
Core mechanism	Requirement register + evidence repository + deterministic checks + human verification + traceable actions.
Prototype approach	Synthetic application records, a small validated requirement set, document metadata and simulated deadlines.

Prepared as a concept and implementation guide. This document describes a proposed design; it does not assert that comparable functionality is absent from existing AICTE systems.

1. Executive summary

Compliance Evidence Intelligence (CEI) is a proposed module within the SIH26262 platform. It helps answer a question that a basic file-upload system cannot answer by itself: **does the submitted evidence satisfy the applicable requirement, is it current, and has the required verification been recorded?**

CEI links requirements, evidence, document metadata, validity periods, verification decisions and follow-up actions. It can identify missing evidence, upcoming expiry, unverified submissions and possible inconsistencies, then explain the finding and route it to the appropriate authorised role.

> **The system supports regulatory processing; it does not replace statutory scrutiny, inspection, legal interpretation or decisions reserved for competent authorities.**

The problem it addresses

- A document being uploaded does not necessarily mean the corresponding obligation is fulfilled.
- Evidence may be missing, out of date, linked to the wrong requirement, awaiting verification or inconsistent with an authorised reference record.
- Follow-up can be fragmented across applications, compliance obligations and document-expiry dates.
- Officers need a clear explanation of what remains unresolved, why it was flagged, who should act, and what the next permitted action is.

What success looks like

Outcome	Possible prototype measure
Fewer overlooked requirements	Percentage of configured requirements with a recorded status and linked evidence or explicit missing state.
Earlier expiry awareness	Percentage of expiring documents flagged within the configured reminder window.
Clearer follow-up	Percentage of issues assigned to a responsible role with a due date and next action.
Traceable verification	Percentage of verification actions with actor, timestamp, outcome and evidence reference.
Reliable alerts	Duplicate-alert rate, failed-delivery rate and successful retry rate in test scenarios.

2. The user problem and proposed value

A conventional document repository stores files and metadata. CEI adds a relationship layer between a requirement and the evidence that is supposed to satisfy it.

Basic portal behaviour	CEI behaviour
Shows that a file was uploaded.	Shows which requirement the file supports and whether verification is pending.
Displays a date entered by a user.	Tracks the source of the date and checks it against a configured validity/reminder policy.
Shows a generic 'pending' status.	Explains the unresolved requirement, supporting evidence, responsible role and next action.
Keeps documents in separate records.	Links evidence, institution, application, obligation, rule version and verification history.
Sends a generic reminder.	Generates a reasoned alert tied to a specific requirement, deadline and status.

Illustrative scenario

Institution ABC has uploaded a lease document and a time-bound undertaking. The lease's recorded expiry date is approaching. The undertaking has been submitted but has no verification outcome recorded. A configured required supporting document is not linked to the relevant requirement.

CEI should display three distinct findings: (1) lease renewal or follow-up is due according to the configured rule; (2) the undertaking awaits authorised verification; and (3) evidence is missing for the specified requirement. It should not label the institution non-compliant solely because an automated check found a potential issue.

Important: this is a synthetic example, not a claim about an actual institution or a verified AICTE-specific document requirement.

Potential users and benefits

- **Institution users:** see upcoming deadlines, missing evidence, submission history and correction requests.
- **Reviewing/compliance officers:** inspect evidence against a requirement checklist and record verification outcomes.
- **Supervisors:** see unresolved obligations, ageing and overdue cases across their authorised scope.
- **Auditors:** inspect read-only histories of evidence, status changes, alerts and verification actions.

3. Functional design: the five components

Component 1 — Requirement register

Represent each applicable requirement as structured data rather than leaving it only in a handbook paragraph or free-text note. A requirement record can include its identifier, description, applicability conditions, expected evidence type, responsible role, verification policy, due-date rule and source citation.

Component 2 — Evidence repository

Store the file securely and maintain metadata that links it to the institution, application, requirement, document type, version, submission date, extracted fields and verification record. Evidence may support more than one requirement only where the configured policy permits it.

Component 3 — Evidence validation

Check	Method and safe interpretation
Missing evidence	Compare applicable requirements with linked submissions; flag no matching evidence.
Expiry / approaching expiry	Compare a verified or reviewed expiry date with the current date and configured reminder windows.
Incomplete document	Check required metadata or use OCR/document parsing to flag possible missing fields/pages for review.
Identifier mismatch	Compare extracted identifiers with authorised records; report a discrepancy, not a fraud conclusion.
Duplicate file	Use file hashes and metadata to flag possible duplicates; allow legitimate repeated submissions.
Verification pending	Check whether an authorised reviewer has recorded the required outcome.

Component 4 — Compliance status engine

Use explicit, mutually understandable states, for example: NOT_SUBMITTED, SUBMITTED_PENDING_VERIFICATION, VERIFIED, ACTION_REQUIRED, EXPIRED, REJECTED_BY_VERIFIER and NOT_APPLICABLE. Define valid transitions and record the reason and actor for each change.

Component 5 — Explanation and action centre

For every finding, display: what was flagged; the supporting evidence or missing link; the rule/source and version; the responsible role; the deadline, if applicable; and the next permitted action. Distinguish an automated flag from an authorised decision.

4. Data model and relationships

A relational database is sufficient for the SIH prototype. A separate graph database is not necessary: evidence-to-requirement relationships can be represented through foreign keys and join tables.

Entity/table	Purpose / example fields
institutions	institution_id, name, category, authorised reference identifiers
applications	application_id, institution_id, application_type, region, stage, status
requirements	requirement_id, title, applicability_rule, evidence_type, verification_required
rule_versions	rule_version_id, source_title, clause_reference, version, effective_date, approval_status
application_requirements	application_id, requirement_id, applicability_status, due_date, rule_version_id
documents	document_id, secure_storage_key, file_hash, document_type, uploaded_by, uploaded_at, version
evidence_links	document_id, application_requirement_id, link_status, linked_at
extracted_metadata	document_id, field_name, extracted_value, confidence, review_status
verification_records	evidence_link_id, reviewer_id, outcome, reason, timestamp
obligations	obligation_id, application_id, type, due_date, status, owner_role
notifications	notification_id, obligation_id, channel, scheduled_at, sent_at, delivery_status
audit_events	actor_id, action, entity_type, entity_id, timestamp, before/after reference

Relationship concept

Institution → Application → Applicable requirement → Evidence link → Document → Verification record. The requirement also references a rule version and source. An obligation may generate reminders and escalation events. This lets an officer navigate from a finding to its evidence and governing configured rule.

Storage and security

- Keep uploaded documents in private object storage; store storage keys and metadata in PostgreSQL rather than exposing public file URLs.
- Use server-side role and record-level authorization; never trust a role or institution identifier supplied only by the browser.
- Record access and verification actions. Protect audit records from ordinary user edits and define retention policies.
- Validate file type and size, scan uploads where feasible, and protect against malicious files and unsafe document parsing.

5. Where CEI fits in ApprovalTwin

CEI is a shared capability used by multiple role-based screens, not a separate website. It connects to the application workflow, compliance register, document store, alert worker and audit trail.

Project area	CEI contribution
Institution dashboard	Missing evidence, upcoming expiry, open obligations, submission status and correction requests.
Officer review screen	Requirement checklist, document viewer, extracted metadata, discrepancy flags and verification controls.
Supervisor dashboard	Open evidence issues, overdue obligations, ageing, assignment and workload overview.
Notification centre	Alerts tied to specific missing evidence, approaching deadlines, overdue actions and verification outcomes.
Workflow engine	Reports whether configured prerequisites are satisfied; blocks a transition only if approved rules explicitly require it.
Audit-log viewer	Trace document uploads, replacements, verification decisions, status changes and notifications.

Example user journey

- An institution uploads a document and links it to a listed requirement.
- The server checks file safety, records metadata and optionally extracts relevant fields using OCR.
- The validation service runs deterministic checks and records any findings.
- The institution sees actionable issues; an authorised officer sees the evidence and review controls.
- The officer verifies, requests correction, or records another permitted outcome with a reason.
- The obligation/deadline worker schedules reminders or escalations based on approved rules.
- The audit history captures who did what, when, against which evidence and rule version.

Do not let OCR or an LLM approve a document, decide statutory eligibility, reject an application, or silently change a regulatory requirement.

6. Technical implementation

Layer	Recommended implementation
Frontend	React + TypeScript + Vite; Tailwind CSS and shadcn/ui for UI; table filters, evidence detail panel and status timeline.
Backend	Node.js + TypeScript + Express; REST endpoints, input validation, server-side authorization and audit-event creation.
Database	PostgreSQL via Supabase or managed PostgreSQL; relational schema, constraints and migrations.
File storage	Private object storage such as Supabase Storage, configured with restricted access and short-lived signed access where appropriate.
OCR	Optional Tesseract or a suitable OCR service; retain extracted value, confidence and review status.
Validation engine	Deterministic TypeScript functions for required evidence, dates, identifiers, applicability and state transitions.
Background worker	Persistent worker plus database-backed jobs for reminders, retry, idempotency and escalation.
Testing	Unit tests for rules, API tests for permissions, integration tests for deadline jobs, and scenario tests with synthetic data.

Suggested API surface

Endpoint (example)	Purpose
GET /applications/:id/requirements	Return applicable requirements and current evidence state.
POST /applications/:id/documents	Upload a file or register its secure storage key and metadata.
POST /evidence-links	Link a document to an applicable requirement, with authorization checks.
POST /evidence/:id/verify	Record an authorised verification outcome and reason.
GET /applications/:id/compliance-summary	Return findings with reason, rule reference, owner and due date.
GET /compliance/expiring	List records due to expire within a configured window for the user's authorized scope.
POST /compliance/:id/actions	Record an authorised follow-up action or correction request.

Keep compliance rules in a dedicated module, separate from Express route handlers. Make checks deterministic and testable; route handlers should enforce authorization, validate inputs, invoke domain logic and return results.

7. OCR and AI: what to use, and what not to use

AI should help extract or organise information, not make unreviewable regulatory decisions. Start with deterministic checks and add OCR only after the end-to-end workflow works.

Capability	Recommended approach
Date extraction	OCR/parser extracts candidate dates; preserve page/location and confidence; require review when uncertain.
Document type classification	Optional model or simple metadata-based suggestion; user confirms classification.
Missing-field detection	Compare extracted fields with a configured checklist; flag uncertainty rather than assume absence.
Identifier comparison	Deterministic comparison against authorised records; normalise formats carefully and explain mismatches.
Regulatory text interpretation	Use approved, versioned rule records. An LLM may assist a reviewer but must cite sources and cannot activate rules on its own.
Compliance status	Rule engine plus authorised verification records; never use an LLM as the final authority.

A useful evidence finding should store its provenance: which rule ran, on what data, when it ran, which document version was checked, and whether a human reviewed the result.

Privacy and security considerations

- Use synthetic data for the hackathon unless real data access is explicitly authorised.
- Avoid sending confidential institutional documents to external AI services without approval and appropriate contractual/security controls.
- Restrict document access by role and institution; log sensitive access where appropriate.
- Treat OCR output as untrusted input and protect against malicious files and prompt injection embedded in documents.
- Use explainable checks with explicit thresholds; do not show false precision such as a '99% compliant' score without validation.

8. Prototype scope and demo script

Build a narrow vertical slice that demonstrates the complete CEI loop. A small, working flow is more persuasive than a large set of disconnected screens.

Minimum viable prototype

- Two synthetic institutions and several synthetic applications.
- Three requirement types: a validity-dated document, a time-bound undertaking, and a required supporting document.
- Requirement applicability configured by application type.
- Document upload or seeded sample files with metadata; OCR is optional for the first demo.
- Evidence-to-requirement linking and explicit verification state.
- Missing-evidence, approaching-expiry and pending-verification findings.
- Institution view, officer verification view and supervisor summary.
- In-app notifications and an audit timeline.

Suggested live demo

Step	Demonstration action
1	Open a synthetic application with three configured requirements.
2	Show one uploaded document, one approaching expiry, one unverified undertaking and one missing evidence item.
3	Open the compliance summary and inspect the reason and rule/source reference for each finding.
4	Upload or link the missing evidence and show the status change.
5	Sign in as an authorised reviewer and record a verification outcome with a reason.
6	Advance a simulated date or use a controlled test clock to trigger a reminder.
7	Open the audit history and show the upload, check, verification and notification events.

Use a clearly labelled test clock or seeded dates rather than waiting for a real expiry. Keep a resettable demo dataset and show the expected result for each step.

9. Testing and evaluation

Test case	Expected result
Applicable requirement has no evidence	Flag missing evidence; show requirement and next action.
Requirement does not apply to this application type	Mark not applicable under the configured rule; do not raise a missing-document alert.
Document expiry is within reminder window	Create one appropriately timed alert and record it.
Document is already expired	Flag expiry and route to configured follow-up; do not automatically reject the application.
Evidence is uploaded but not verified	Keep state as pending verification; do not mark fulfilled.
Identifier mismatch is detected	Show compared values and source; request human review.
Unauthorized user tries to view another institution's document	Deny access and record the security event where configured.
Background worker restarts	Pending jobs remain durable and are retried safely.
Same reminder job runs twice	Idempotency prevents duplicate user notifications.
Document is replaced with a new version	Preserve the old version and link verification to the correct version.
Rule or requirement configuration changes	Record the version used by the check; require authorised configuration and review.

Metrics to report

- Requirement coverage: share of applicable requirements with an explicit evidence/status record.
- Finding precision on the controlled test set: share of generated findings that match expected results.
- Expiry alert timeliness and duplicate-alert rate.
- Verification traceability: share of outcomes with actor, timestamp, reason and evidence version.
- Task resolution time in simulated scenarios.

Avoid claiming real-world efficiency gains from a synthetic demo. Report measured prototype results and describe production impact as a hypothesis requiring field validation.

10. Risks, safeguards and limitations

Risk	Mitigation
Incorrect requirement configuration	Require source citations, versioning, review and approval before activation.
OCR extracts a wrong date or identifier	Store confidence and source location; require human review for uncertain or consequential findings.
False non-compliance flag	Use neutral statuses such as 'potential issue' or 'review required'; explain evidence and allow correction.
Incorrect applicability	Use explicit category and effective-date rules; handle exceptions and ambiguity through authorised review.
Duplicate or missed alerts	Persistent jobs, idempotency keys, retry policies, monitoring and alert history.
Unauthorized access to documents	Private storage, server-side authorization, least privilege, secure file handling and audit logs.
Rule changes affect cases incorrectly	Version rule configurations, preview impact, approve changes and retain historical decisions.
Overreliance on automation	Keep statutory scrutiny, inspections, verification and decisions with competent authorities.

Known limitations

- CEI can only validate against requirements and reference data that have been accurately configured and authorised.
- It cannot independently establish the legal validity of every document or resolve ambiguous regulatory interpretation.
- OCR may fail on low-quality scans, handwriting, varied formats or multilingual documents.
- The prototype cannot prove integration with AICTE production systems without authorised access and interfaces.
- The existence or absence of similar capabilities in current AICTE systems must be verified before claiming novelty.

11. Build plan

Milestone	Deliverable	Exit criterion
1. Domain model	Requirement types, states, applicability and roles	A requirement can be linked to an application and rule source.
2. Data and storage	PostgreSQL schema and private file storage	Upload, access control and evidence links work.
3. Deterministic checks	Missing evidence, expiry and verification checks	Unit tests cover positive, negative and edge cases.
4. Role-based UI	Institution, reviewer and supervisor views	Each role sees only authorised records/actions.
5. Alerts and audit	Background jobs, notification centre and event history	Retries work; duplicate alerts are prevented.
6. Optional OCR	Date/identifier extraction and review UI	Extraction results are traceable and never auto-approve evidence.
7. Demo hardening	Seeded data, reset, test clock and script	Demo can run end-to-end repeatedly.

Suggested priority

Must have: requirement register, evidence links, missing/expiry/pending-verification checks, role-based access, notifications and audit trail.

Should have: document versioning, configurable reminder windows, rule-source references and a supervisor summary.

Could have: OCR, suggested document classification, cross-document discrepancy detection and richer analytics.

Do later: predictive ML, broad LLM-based document interpretation, complex external integrations and large-scale process mining.

12. Novelty positioning and final recommendation

The novelty should not be described as 'we use OCR' or 'we check document expiry'. Those are established capabilities. A stronger, more defensible proposition is the integration of requirement-to-evidence traceability, validity and consistency checks, authorised verification, actionable explanations and a complete audit trail in the AICTE approval and post-approval compliance context.

Suggested one-line pitch

“Compliance Evidence Intelligence connects every applicable AICTE requirement to its supporting evidence, checks what can be verified automatically, explains unresolved gaps, and routes the next action to an authorised user—with every finding and verification traceable to its source.”

What we can responsibly claim

- The prototype demonstrates evidence-to-requirement mapping and explainable compliance checks.
- It distinguishes uploaded evidence from verified fulfilment.
- It can monitor configured validity dates and create traceable alerts.
- It supports human oversight and role-based access.
- It is designed to integrate with authorised systems, subject to available interfaces and approvals.

What we should not claim without evidence

- That no existing AICTE system already performs these functions.
- That automated checks establish legal validity or replace a competent authority.
- That the prototype reduces real-world processing time before it has been evaluated in a representative setting.
- That an AI-generated risk or compliance score is accurate without a validated test dataset.

Final recommendation

Build CEI as a small, explainable, rules-driven module first. Demonstrate one application end-to-end: applicable requirement → evidence submission → automated checks → human verification → deadline alert → audit history. That will show both practical value and engineering maturity. Treat OCR as an enhancement, not the foundation.

Reference and validation note

The design is based on the supplied SIH Problem Statement 26262 description and the concept developed in this conversation. Before implementation, validate every requirement, deadline, applicability condition and role against the currently applicable official AICTE Approval Process Handbook, amendments, approval conditions and authorised procedures. Public descriptions alone do not establish what capabilities are or are not present in existing internal systems.